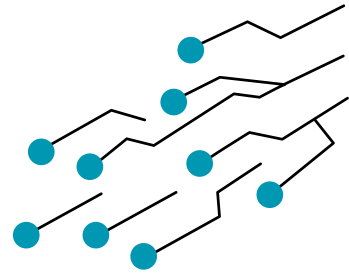


NOM
DU PROJET :



SPNX-16 :

(SUBSTITUTION-PERMUTATION
NETWORK EXTENDED – 16-BIT)

CAHIER DES CHARGES

Club :

SECURINETS FST

Élaboré par :
Eya Karabaka

Encadré par :
Chahine Ben Salah

Projet : Chiffrement Par Bloc

SOMMAIRE

01 / Présentation et Objectifs du Projet

02 / Identité et Nom de l'Algorithme — SPNX-16

03 / Spécifications Techniques Détaillées

04 / Architecture Cryptographique

05 / Modèle de Menace et Positionnement Sécuritaire

06 / Fonctionnalités Attendues Par Module

07 / Architecture Logicielle et Organisation des Fichiers

08 / Stratégie de Validation et Tests

09 / Annexes

1. PRÉSENTATION ET OBJECTIFS DU PROJET

1.1 CONTEXTE GÉNÉRAL

La cryptographie symétrique constitue un pilier fondamental de la sécurité des communications numériques modernes, notamment dans les protocoles HTTPS, les systèmes de messagerie chiffrée, le stockage sécurisé des données et les mécanismes d'authentification Wi-Fi. Le standard de référence actuel, l'Advanced Encryption Standard (AES), repose sur une structure bien définie appelée Réseau de Substitution-Permutation (SPN).

Dans le cadre de ce projet, il est demandé de concevoir et d'implémenter une version simplifiée de cette architecture, à échelle réduite mais respectant ses principes fondamentaux. L'implémentation sera réalisée en Python pur, sans recours à des bibliothèques cryptographiques externes.

Cette contrainte pédagogique vise à garantir une compréhension approfondie des mécanismes internes de la cryptographie symétrique. Chaque opération est ainsi explicitement définie et implémentée, permettant une manipulation directe des bits et une observation claire du fonctionnement interne du chiffrement.

1.2 OBJECTIFS PÉDAGOGIQUES

- Manipuler des opérations bas niveau sur les bits : XOR, rotation circulaire, découpage en nibbles, recomposition d'octets.
- Comprendre et concevoir des primitives cryptographiques originales : S-Box non-linéaire bijective, table de permutation diffusante, key schedule par rotation.
- Mesurer empiriquement l'effet avalanche plutôt que de l'admettre théoriquement.

- Quantifier la résistance d'un système à une attaque exhaustive et comprendre pourquoi la taille de clé est le paramètre de sécurité central.
- Développer une interface graphique complète intégrant chiffrement, analyse et attaque.

1.3 OBJECTIFS TECHNIQUES

- Livrer un moteur de chiffrement/déchiffrement réversible, déterministe et entièrement documenté.
- Développer un module d'analyse de sécurité avec visualisations graphiques (effet avalanche, distribution statistique).
- Implémenter une attaque par force brute complète sur l'espace de clés 2^{16} avec extrapolation théorique.
- Intégrer tous les modules dans une interface graphique unifiée.

1.4 AVERTISSEMENT SÉCURITAIRE



AVERTISSEMENT :

- Le système SPNX-16 est conçu dans un cadre strictement académique et pédagogique. En raison de l'utilisation d'une clé de 16 bits (soit 65 536 combinaisons possibles), il présente une vulnérabilité importante aux attaques par force brute et peut être compromis en quelques secondes par des moyens de calcul modernes, comme le démontrera le module d'attaque inclus dans ce projet.
- Par conséquent, SPNX-16 ne doit en aucun cas être utilisé pour la protection de données réelles ou dans un environnement de production. Il s'agit uniquement d'un prototype destiné à l'étude et à la compréhension des principes fondamentaux de la cryptographie symétrique.

2. IDENTITÉ ET NOM DE L'ALGORITHME : SPNX-16

2.1 ORIGINE DU NOM

Le nom SPNX-16 n'est pas choisi au hasard. Chaque composante a un sens technique précis, à l'image du nommage des vrais standards cryptographiques (AES-128, ChaCha20, SHA-256) où le chiffre final communique immédiatement le paramètre de sécurité le plus critique.

Composante	Signification	Justification
S	Substitution	Référence directe à la S-Box : première primitive fondamentale de l'architecture SPN
P	Permutation	Référence à la table de permutation : deuxième primitive, responsable de la diffusion
N	Network	Réseau : les primitives S et P sont chaînées en réseau de rounds successifs
X	eXtended	Marque la volonté de dépasser l'implémentation minimale : module d'analyse, attaque, GUI, DDT
16	16 bits de clé	Paramètre de sécurité principal : $2^{16} = 65\,536$ clés . c'est le chiffre que tout le projet interroge (la longueur de la clé)

2.2 POURQUOI LE CHIFFRE 16 PLUTÔT QUE 8 ?

Dans l'architecture du système, deux paramètres numériques fondamentaux sont définis : la taille de bloc (8 bits) et la taille de clé (16 bits).

Le choix de mettre en avant la valeur « 16 » dans la dénomination du système s'explique par le fait que la sécurité contre les attaques par force brute dépend directement de la taille de la clé, et non de celle des blocs.

Cette convention suit les standards cryptographiques modernes, tels que l'AES-128, qui est nommé d'après la longueur de sa clé (128 bits) et non la taille de ses blocs internes. Ainsi, le nom SPNX-16 adopte la même logique de représentation.

2.3 POURQUOI CONSTRUIRE SON PROPRE ALGORITHME PLUTÔT QU'UTILISER L'EXISTANT ?

C'est la question que tout développeur pragmatique poserait : pourquoi reconstruire from scratch quelque chose qui existe déjà, en mieux, depuis des décennies ? La réponse n'est pas technique, elle est épistémologique. Ce qu'on construit soi-même, on le comprend d'une façon radicalement différente de ce qu'on utilise.

Utiliser une bibliothèque existante	Construire SPNX-16 from scratch
On appelle aes.encrypt() sans savoir ce qui se passe à l'intérieur	On écrit chaque XOR, chaque rotation, chaque substitution et on sait exactement pourquoi
On fait confiance à la boîte noire. Si elle échoue, on ne comprend pas pourquoi	On peut localiser, comprendre et corriger chaque faille, parce qu'on a tout construit
"L'effet avalanche" reste un concept abstrait dans une doc	On mesure l'effet avalanche bit par bit, on le voit changer quand on modifie la S-Box
On ne peut pas casser sa propre implémentation — elle est correcte par définition	On lance sa propre attaque par force brute contre son propre algo et on comprend viscéralement ce que signifie 2^{16} vs 2^{128}
Compétence acquise : savoir utiliser une API	Compétence acquise : comprendre les fondements de la cryptographie symétrique moderne

3. SPÉCIFICATIONS TECHNIQUES DÉTAILLÉES

3.1 SPÉCIFICATIONS TECHNIQUES EN DÉTAIL

Paramètre	Valeur	Remarque
Taille de bloc	8 bits (1 octet)	Un caractère ASCII par bloc
Taille de clé principale	16 bits	65 536 clés possibles — espace exhaustible
Nombre de rounds	4	Rounds 1–3 identiques, round 4 avec whitening final
Nombre de sous-clés	5 (K1 à K5)	K1–K4 : XOR d'entrée de round / K5 : whitening final
Structure S-Box	4 bits → 4 bits	2 S-Box identiques appliquées en parallèle (nibble haut et bas)
Permutation	8 bits → 8 bits	Appliquée uniquement aux rounds 1 à 3
Encodage texte	UTF-8 → octets	Géré par le module de padding
Langage	Python ≥ 3.8	Aucune lib crypto externe
GUI	Tkinter (défaut)	Inclus nativement — justifié en section 6.4
Visualisations	matplotlib	Pour les graphiques d'effet avalanche

3.2 STRUCTURE DES ROUNDS EN DÉTAIL

Round	Étape 1	Étape 2	Étape 3
Round 1	XOR avec K1	Substitution (2× S-Box 4 bits)	Permutation des bits
Round 2	XOR avec K2	Substitution (2× S-Box 4 bits)	Permutation des bits
Round 3	XOR avec K3	Substitution (2× S-Box 4 bits)	Permutation des bits
Round 4 (final)	XOR avec K4	Substitution (2× S-Box 4 bits)	XOR K5 – whitening final (pas de permutation)

4. ARCHITECTURE CRYPTOGRAPHIQUE

4.1 LA S-BOX DE SPNX-16

La S-Box constitue la principale composante de non-linéarité de l'algorithme SPNX-16. Son rôle est d'introduire une transformation non linéaire indispensable au renforcement des propriétés cryptographiques du système. En son absence, le chiffrement se limiterait à une succession d'opérations linéaires (XOR et permutation), le rendant vulnérable à des attaques fondées sur des méthodes d'algèbre linéaire.

Par conséquent, la conception de cette S-Box doit satisfaire un ensemble de critères cryptographiques rigoureux afin de garantir un niveau de sécurité conforme aux objectifs du projet. Les principaux critères retenus sont présentés dans les sections suivantes.

► TABLE S-BOX PROPOSÉE :

```
SBOX = [0x6, 0xB, 0x9, 0x0, 0xC, 0xA, 0x5, 0xF,  
        0x3, 0xE, 0x1, 0xD, 0x4, 0x7, 0x8, 0x2]  
# index = nibble entrant → valeur = nibble sortant
```

Entrée	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Sortie	6	B	9	0	C	A	5	F	3	E	1	D	4	7	8	2

► CRITÈRES DE CONCEPTION ET VÉRIFICATIONS :

Critère	Définition	Statut
Bijektivité	Toutes les valeurs de sortie sont distinctes : condition nécessaire pour l'inversibilité au déchiffrement	Vérifié
Absence de point fixe	Aucune entrée x telle que $SBOX[x] = x$: évite une faiblesse statistique triviale	Vérifié
Absence de point fixe opposé	Aucune entrée x telle que $SBOX[x] = \sim x$ (sur 4 bits) : évite une corrélation directe	Vérifié
Non-linéarité	Aucune relation linéaire simple (type $SBOX[x] = x \text{ XOR constante}$) ne relie entrée et sortie	À vérifier via DDT
S-Box inverse	Construire automatiquement INV_SBOX depuis $SBOX$ — utilisée au déchiffrement	Généré par code

► S-BOX INVERSE (POUR LE DÉCHIFFREMENT) :

```
INV_SBOX = [0] * 16
for i, v in enumerate(SBOX):
    INV_SBOX[v] = i    # INV_SBOX[SBOX[x]] == x pour tout x
```

4.2 LA TABLE DE PERMUTATION

La permutation constitue le mécanisme de diffusion de l'algorithme SPNX-16. Son rôle est de redistribuer les bits du bloc après l'étape de substitution afin de propager, au fil des rounds, les modifications introduites par la S-Box à l'ensemble des données traitées. Cette diffusion favorise l'interdépendance des différentes parties du bloc et renforce la résistance du chiffrement aux attaques cryptanalytiques.

En l'absence de cette étape, les deux nibbles évolueraient de manière indépendante à chaque round, réduisant le chiffrement à deux chaînes de substitution parallèles sans interaction. Une telle architecture diminuerait considérablement la complexité du système et compromettrait son niveau de sécurité.

► **TABLE DE PERMUTATION PROPOSÉE :**

Position entrée	0	1	2	3	4	5	6	7
Position sortie	6	2	7	1	3	0	4	5

► **JUSTIFICATION DU DESIGN :**

Le principe de conception : sur les 4 bits du nibble haut (positions 0–3), deux bits restent dans le nibble haut et deux basculent vers le nibble bas. Symétriquement pour le nibble bas. Résultat : après une seule permutation, chaque nibble de sortie contient un mélange de bits issus des deux nibbles d'entrée. Après 3 rounds successifs, chaque bit de sortie dépend statistiquement de plusieurs bits d'entrée différents.

4.3 LE KEY SCHEDULE : DÉRIVATION DES SOUS-CLÉS

Le Key Schedule produit 5 sous-clés de 8 bits à partir de la clé principale de 16 bits, par rotations circulaires gauche (RoL) successives. L'idée : à chaque fois, on fait tourner la clé d'un nombre de positions différent, puis on extrait les 8 premiers bits du résultat.

Sous-clé	Dérivation	Décalage RoL	Utilisation
K1	bits[0:8] de la clé principale	Aucun	XOR Round 1
K2	bits[8:16] de la clé principale	Aucun	XOR Round 2
K3	RoL(clé, 3) → bits[0:8]	3 positions	XOR Round 3
K4	RoL(clé, 6) → bits[0:8]	6 positions	XOR Round 4
K5	RoL(clé, 9) → bits[0:8]	9 positions	Whitening final (après substitution du Round 4)

► IMPLÉMENTATION PYTHON DU RoL :

```
def roll6(val, n):
    """Rotation circulaire gauche sur 16 bits."""
    n = n % 16
    return ((val << n) | (val >> (16 - n))) & 0xFFFF
```

4.4 POURQUOI LE DERNIER ROUND N'A-T-IL PAS DE PERMUTATION ?

Le dernier round de SPNX-16 ne comporte volontairement aucune étape de permutation. Ce choix de conception repose sur le fait qu'une permutation est une transformation publique et entièrement déterministe, dont la description est connue de tout utilisateur ou attaquant. En outre, elle modifie uniquement la position des bits sans en altérer les valeurs.

Par conséquent, si une permutation constituait la dernière opération du chiffrement, elle pourrait être annulée directement par l'application de sa permutation inverse, sans nécessiter la moindre connaissance de la clé secrète. Une telle opération n'apporterait donc aucun renforcement de la sécurité en fin de processus.

En cryptographie, une opération de diffusion n'est réellement efficace que lorsqu'elle est suivie d'une transformation dépendante de la clé. Cette approche est également adoptée par les chiffrements modernes, notamment l'algorithme AES, dont le dernier round ne comporte pas l'étape MixColumns, afin que la transformation finale reste protégée par une opération cryptographique liée à la clé.

Dans SPNX-16, cette exigence est satisfaite par l'application d'un XOR final de blanchiment (Final Whitening) utilisant la sous-clé K5, garantissant que la dernière transformation appliquée au bloc demeure directement dépendante de la clé secrète.

5. MODÈLE DE MENACE ET POSITIONNEMENT SÉCURITAIRE

Cette section définit le modèle de menace retenu pour SPNX-16 ainsi que les objectifs de sécurité associés. Elle précise les types d'attaques contre lesquels le système est conçu pour offrir une protection et identifie, de manière explicite, les limites inhérentes à son architecture et à ses choix de conception.

L'objectif est de délimiter clairement le périmètre de sécurité de SPNX-16 en distinguant les propriétés cryptographiques effectivement assurées de celles qui relèvent de mécanismes ou d'exigences dépassant le cadre du présent projet.

5.1 TYPES D'ATTAQUES

Type d'attaque	SPNX-16 résiste ?	Explication
Force brute exhaustive (2^{16} clés)	Non	Espace de clé intentionnellement petit. Cassé en quelques secondes : démontré par le module d'attaque
Analyse de fréquence simple	Oui (relatif)	Les rounds multiples + diffusion cassent la corrélation statistique directe lettre → symbole
Attaque à clair connu (Known Plaintext)	Partiel	Possible avec analyse différentielle avancée : hors périmètre mais mentionné
Cryptanalyse différentielle / linéaire	Non évalué	La DDT de la S-Box sera calculée (module analyse) mais pas exploitée en attaque
Attaque par canal auxiliaire (timing)	Non traité	Implémentation non constant-time — limite assumée et documentée

5.2 COMPARAISON AVEC AES-128

Critère	SPNX-16 (académique)	AES-128 (standard réel)
Taille de clé	16 bits	128 bits
Espace de clés	65 536	$3,4 \times 10^{38}$
Temps force brute (1M clés/s)	< 0,1 seconde	$\sim 10^{25}$ années
Taille de bloc	8 bits	128 bits
Nombre de rounds	4	10 (AES-128)
Architecture	SPN miniature	SPN industriel complet
Usage	Pédagogie / recherche	Production mondiale

6. FONCTIONNALITÉS ATTENDUES PAR MODULE

6.1 MODULE CORE : CHIFFREMENT / DÉCHIFFREMENT

- **chiffrer(message: str, cle: int) -> bytes** : convertit le message en blocs de 8 bits, applique le padding, exécute les 4 rounds bloc par bloc.
- **dechiffrer(message_chiffre: bytes, cle: int) -> str** : applique la séquence inverse (rounds en ordre inverse, S-Box inverse, permutation inverse, XOR).
- **key_schedule(cle: int) -> list[int]** : génère K1 à K5 par rotations circulaires.
- **Gestion du padding** : norme PKCS#7 adaptée à 8 bits — ajout d'octets de valeur N pour compléter le dernier bloc, N étant le nombre d'octets ajoutés. Retiré automatiquement au déchiffrement.

6.2 MODULE ANALYSIS : SÉCURITÉ

- **Mesure de l'effet avalanche** : chiffrer M et M' (M avec exactement 1 bit modifié), compter les bits différents dans les sorties, exprimer en pourcentage.
- **Exécution statistique** sur un grand nombre de messages et clés aléatoires (pas un essai isolé — un seul test ne vaut rien statistiquement).
- **Visualisation** : histogramme de la distribution du nombre de bits modifiés (matplotlib), avec ligne de référence à 50 %.

- **Calcul de la Difference Distribution Table (DDT) de la S-Box** : pour chaque différence d'entrée Δx , mesure la distribution des différences de sortie Δy . Permet d'objectiver la non-linéarité.
- **Calcul de la Linear Approximation Table (LAT)** : mesure la corrélation linéaire entre bits d'entrée et bits de sortie de la S-Box.

6.3 MODULE ATTACK : FORCE BRUTE

- Parcours exhaustif des 65 536 clés possibles pour retrouver la clé ayant produit un chiffré donné, avec un couple (clair, chiffré) connu.
- Chronométrage précis (time.perf_counter), calcul du débit en clés/seconde.
- **Extrapolation théorique** : à partir du débit mesuré, calcul du temps nécessaire pour casser 32, 64 et 128 bits de clé — mise en perspective avec AES-128 pour donner du sens au chiffre.
- **Attaque par dictionnaire optionnelle** : tester d'abord un sous-ensemble de clés « courantes » (séquences simples : 0x0000, 0xFFFF, 0xAAAA...) pour illustrer l'importance d'une clé aléatoire.

6.4 MODULE GUI — INTERFACE GRAPHIQUE

Bibliothèque choisie : Tkinter

Justification : Tkinter est inclus nativement dans Python 3, sans aucune dépendance externe à installer. Ce choix est cohérent avec la contrainte principale du projet (zéro bibliothèque externe non justifiée) et garantit que l'application fonctionne sur n'importe quelle machine Python sans configuration supplémentaire.

Onglet	Fonctionnalités
Chiffrer / Déchiffrer	Zone de saisie message, champ clé (16 bits), boutons Chiffrer / Déchiffrer, affichage résultat en binaire et hexadécimal
Analyse	Lancement de l'analyse d'effet avalanche, saisie du nombre d'itérations, affichage graphique matplotlib intégré + résumé statistique
Attaque	Saisie du couple clair/chiffré connu, lancement force brute, barre de progression, affichage clé trouvée + temps écoulé + extrapolation
À propos	Description de SPNX-16, avertissement sécuritaire (rappel du point 1.4), schéma des rounds, crédits équipe

7. ARCHITECTURE LOGICIELLE ET ORGANISATION DES FICHIERS

7.1 ARBORESCENCE DE FICHIERS

```
spnx16/
├── core/
│   ├── __init__.py
│   ├── sbbox.py           # S-Box, S-Box inverse, vérification bijectivité
│   ├── permutation.py    # Table de permutation et son inverse
│   ├── key_schedule.py   # Génération des sous-clés K1-K5 (RoL)
│   ├── cipher.py         # chiffrer() / déchiffrer(), logique des rounds
│   └── padding.py        # Gestion du padding PKCS#7 adapté
├── analysis/
│   ├── avalanche.py     # Mesure effet avalanche
│   ├── ddt.py           # Difference Distribution Table (DDT) – bonus
│   └── plots.py          # Visualisations matplotlib
├── attack/
│   └── bruteforce.py     # Attaque exhaustive + extrapolation théorique
├── gui/
│   └── app.py            # Interface Tkinter – 4 onglets
├── tests/
│   ├── test_reversibilite.py
│   ├── test_determinisme.py
│   ├── test_sensibilite_cle.py
│   ├── test_avalanche.py
│   └── test_sbbox.py     # Bijectivité, point fixe, point fixe opposé
├── rapport/
│   └── rapport_final.pdf # Justifications, résultats, analyse
├── main.py               # Point d'entrée : lance la GUI
└── README.md             # Installation, usage, description
```

7.2 PRINCIPES DE CODE

- Chaque fonction doit avoir une docstring Python (description, paramètres, valeur de retour).
- Chaque bloc logique non-trivial doit avoir un commentaire inline.

- Aucun import cryptographique externe : hashlib, secrets, cryptography, pycryptodome sont interdits pour le cœur de l'algo.
- matplotlib et tkinter sont autorisés pour GUI et visualisations.

8. STRATÉGIE DE VALIDATION ET TESTS

8.1 TESTS OBLIGATOIRES

Test	Méthode	Condition de succès
Réversibilité	Pour N messages et clés aléatoires : $\text{dechiffrer}(\text{chiffrer}(M,K),K) == M$	100% de réussite sur au moins 10 000 cas
Déterminisme	Deux appels successifs à $\text{chiffrer}(M,K)$ doivent être strictement identiques	100% de réussite
Sensibilité clé	Pour $K \neq K'$, vérifier $\text{chiffrer}(M,K) \neq \text{chiffrer}(M,K')$ sur un échantillon	100% de réussite sur 1000 paires de clés
Effet avalanche	Sur un grand échantillon, la moyenne du % de bits modifiés doit être proche de 50%	Entre 40% et 60% (tolérance de $\pm 10\%$)

8.2 TESTS BONUS

Test	Description
Bijektivité S-Box	Vérifier automatiquement que $\text{len}(\text{set}(\text{SBOX})) == 16$ (toutes les sorties distinctes)
Point fixe S-Box	Vérifier qu'aucun i ne satisfait $\text{SBOX}[i] == i$
Point fixe opposé	Vérifier qu'aucun i ne satisfait $\text{SBOX}[i] == (\sim i \ \& \ 0xF)$
Cas limites padding	Message vide, message d'un seul caractère, message exactement multiple de 8 bits
Calcul DDT S-Box	Calculer et afficher la Difference Distribution Table pour objectiver la non-linéarité

9. ANNEXES

Terme	Définition
SPN	Substitution-Permutation Network — architecture d'algorithme de chiffrement par blocs alternant substitution non-linéaire et permutation diffusante
S-Box	Boîte de substitution. Table de correspondance non-linéaire qui remplace une valeur d'entrée par une valeur de sortie différente
Nibble	Groupe de 4 bits. Un octet (8 bits) est composé de deux nibbles : nibble haut (bits 0-3) et nibble bas (bits 4-7)
Bijektivité	Propriété d'une fonction où chaque valeur d'entrée produit une valeur de sortie unique — condition nécessaire pour l'inversibilité
Diffusion	Propriété selon laquelle modifier 1 bit d'entrée propage son effet sur de nombreux bits de sortie
Whitening	XOR additionnel avec une sous-clé, appliqué en début ou en fin d'algorithme, pour empêcher des attaques exploitant la structure interne
Effet avalanche	Critère de qualité : modifier 1 bit d'entrée doit modifier environ 50% des bits de sortie
RoL	Rotation circulaire gauche (Rotate Left) — décalage des bits vers la gauche où les bits sortant réapparaissent à droite
DDT	Difference Distribution Table — table mesurant la non-linéarité d'une S-Box en analysant la propagation des différences
XOR ($\oplus$)	Opération bit à bit : $0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$, $1 \oplus 1 = 0$. Propriété clé : $A \oplus B \oplus B = A$ (auto-inverse)
Force brute	Attaque exhaustive consistant à tester toutes les clés possibles jusqu'à trouver la bonne
Padding	Technique d'ajout d'octets supplémentaires pour compléter le dernier bloc à la taille requise